> **Archivo:** `entrega/documento-entrega.md`
> **Objetivo:** Documento de entrega · Taller A2A (Curso 2 · Sesión 1)
> **Grupo del curso:** Fabio Durán - Gustavo Reyes - Ezequiel Muñoz
> **Creación:** 20261110 1941 · **Última modificación:** 20261111 1205

Documento de entrega · Taller A2A (Curso 2 · Sesión 1)

> Documento completo. El PDF se genera con `uv run tools/exportar_pdf.py`. Las capturas `01-saltos.png` / `03-tunel.png` se generaron desde la corrida real (reemplazables por capturas de pantalla manuales). Fabio y Gustavo completan sus blueprints (plantillas ya creadas).

1. Identificación

- **Curso 2 · Sesión 1** — Taller "tu skill es un agente y habla por A2A".
- **Grupo:** NN
- **Integrantes:**
- Fabio Durán
- Gustavo Reyes
- Ezequiel Muñoz
- **Repositorio (público):** https://github.com/etmunoz/tarea_1_curso_2

2. Blueprints (uno por integrante)

- **Ezequiel · IA Engineer & Data Scientist** -> `blueprint-ezequiel.md`
- **Fabio** -> `blueprint-fabio.md` (plantilla; la completa Fabio)
- **Gustavo** -> `blueprint-gustavo.md` (plantilla; la completa Gustavo)

3. Mi agente (Paso 3)

- Paquete `src/agente_ezequiel/`: herramientas `recomendar_tecnica`, `metricas_recomendadas`, `elegir_modelo` (datos simulados), prompt separado y grafo `create_agent` registrado en `langgraph.json` como `agente_ezequiel`.
- Además el grupo levantó dos agentes más en esta misma repo: `agente_tomas` (Reiki Master con RAG sobre Redis) y `agente_cocina` (recetas chilenas con costeo en SQLite).
- **Prompt usado** para crearlo: `prompts_ejecutados/04-crear-agente-ezequiel.md`.

4. Probarlo solo (Paso 4)

- Comandos y salida: `respuestas/04-crear-agente-ezequiel.md`.
- Evidencia: `evidencias/01-saltos.png` (ronda real por el coordinador, generada desde la corrida).

5. La ficha / Agent Card (Paso 5)

- Las fichas se aplican con `tools/configurar_fichas.py` (hace `PATCH /assistants/<id>` con `name` + `description` de `src/fichas.py`).
- Evidencia: `evidencias/02-ficha-agente_ezequiel.json` (y además `02-ficha-agente_tomas.json`, `02-ficha-agente_cocina.json`, `02-ficha-agente_viajes.json`).

6. Salir a internet · Cloudflare (Paso 6)

- Túnel: `cloudflared tunnel --url http://127.0.0.1:2024` (HTTP, sin abrir puertos).
- La dirección pública `https://...trycloudflare.com` es efímera (cambia en cada corrida); se comparte entre equipos al momento de la ronda.
- Evidencia: `evidencias/03-tunel.png` (ficha del agente servida por la URL pública de Cloudflare).

7. Sumar a los compañeros · coordinador (Paso 7)

- Coordinador/supervisor: `src/coordinador/` (enruta por ficha; `URLS_AGENTES`). Marca cada agente como local o remoto y comparte caché de fichas.
- Catálogo real de agentes: `evidencias/04-catalogo-mac.txt` y `evidencias/04-catalogo-windows.txt` (prueba bidireccional Mac <-> Windows, todo por Cloudflare, en redes distintas).
- Agentes ajenos a sumar por los compañeros: ver `companeros.md`.

8. La ronda de preguntas (Paso 8)

Ronda real registrada en `evidencias/05-ronda-mac.md` y `05-ronda-windows.md`:

Pregunta	A quién enrutó el supervisor	¿Estuvo bien?
¿Qué técnica uso si tengo desbalance de clases?	[Agente de Ezequiel] (local)	sí
¿Qué símbolos se aprenden en el nivel 2 de Reiki?	[Agente de Tomás] (remoto, por Cloudflare)	sí
¿Cuánto cuesta un pastel de choclo para 4 personas?	[Agente de cocina chilena] (local)	sí
¿Quién ganó el mundial de 1962?	sin_agente (fuera de dominio)	sí (lo rechaza bien)

- Comentario: el supervisor eligió correctamente por ficha; derivó al agente remoto (Tomás) por Cloudflare sin que el usuario note la diferencia (A2A transparente) y rechazó con cortesía lo que ningún agente cubre.
- Evidencia: `evidencias/05-ronda-mac.md`, `evidencias/05-ronda-windows.md`.

9. Cómo correr el proyecto

```
uv sync
cp .env.example .env          # pega tu GOOGLE_API_KEY
docker compose up -d          # Redis para el RAG de Tomás
uv run langgraph dev          # publica los grafos en :2024
uv run python tools/configurar_fichas.py   # aplica las fichas (Agent Cards)
uv run pytest                 # pruebas
```

- Coordinador / app web / túnel: ver los scripts `scripts/mac_*.sh` y `scripts/win_*.ps1` (plug and play por equipo) y la guía `taller-a2a/`.

10. Robustez (resiliencia + guardrails)

Dos implementaciones técnicas del coordinador (prompt 44):

- **Agente remoto no disponible:** `src/coordinador/a2a.py` convierte conexión caída / timeout / HTTP inválido / JSON ilegible en `AgenteNoDisponible`; el catálogo se arma igual marcando al caído, y el supervisor devuelve un estado controlado («... no está disponible temporalmente ... los demás continúan operativos»).
- **Guardrails** (`src/coordinador/guardrails.py`): **entrada** (`guardia_entrada`: vacías/largas/inyección), **salida** (`guardia_salida`, nodo común a todo camino: redacción de credenciales + tope de longitud + no vacío) y **A2A** (respuesta remota como dato, no instrucción).
- **Allow-list de agentes** (`A2A_AGENTES_PERMITIDOS`): lista blanca explícita de los grafos que el coordinador puede invocar (sus "tools"); un túnel colado no se ejecuta.
- Pruebas: `tests/test_resiliencia_a2a.py`, `tests/test_guardrails.py`, `tests/test_salida_y_allowlist.py`.
- Evidencia (con un túnel de compañero caído): `evidencias/06-robustez-mac.md`.

11. Evidencias incluidas

Ver carpeta `evidencias/` (lista y requisitos en `evidencias/README.md`):

- `02-ficha-agente_*.json` — Agent Cards aplicadas.
- `04-catalogo-mac.txt`, `04-catalogo-windows.txt` — catálogo del coordinador en cada equipo.
- `05-ronda-mac.md`, `05-ronda-windows.md` — ronda de preguntas en cada equipo.
- `01-saltos.png`, `03-tunel.png` — generadas desde la corrida real (coordinador y túnel).